

JavaScript **this**, **call**, **apply**, and **bind** - Complete In-Depth Notes

Table of Contents

1. [Introduction](#)
 2. [Strict Mode vs Non-Strict Mode](#)
 3. [Global Objects in Different Environments](#)
 4. [The **this** Keyword - Complete Breakdown](#)
 5. [**call**, **apply**, and **bind** Methods](#)
 6. [Arrow Functions and **this**](#)
 7. [Practical Examples and Use Cases](#)
 8. [Summary and Quick Reference](#)
-

Introduction

Why These Concepts Matter

- **Interview Importance:** **this**, **call**, **apply**, and **bind** are favorite interview topics
- **Real-world Usage:** While rarely used in daily coding, understanding these concepts is crucial
- **Senior-level Thinking:** These concepts help develop advanced problem-solving approaches

Study Strategy

Important Note: You will likely forget these concepts if not used regularly. **Solution:** Revise these notes 1-2 days before your interview to brush up on all concepts.

Strict Mode vs Non-Strict Mode

What is Strict Mode?

Strict mode is a feature introduced in ECMAScript 5 that enforces stricter parsing and error handling in JavaScript code.

Why Strict Mode Was Introduced

Problem with Old JavaScript: JavaScript was very "forgiving" - it ignored many errors and let code run even with mistakes.

Example 1: Undeclared Variables

Without Strict Mode:

```
a = 10;
b = 20;
console.log(a); // 10
console.log(b); // 20
```

Problem: Variable `b` was never declared with `let`, `const`, or `var`, but the code still runs. This becomes a **source of bugs**.

With Strict Mode:

```
"use strict";
a = 10;
b = 20; // ERROR: b is not defined
console.log(a);
console.log(b);
```

Result: Immediate error → "b is not defined"

Example 2: Duplicate Parameter Names

Without Strict Mode:

```
function greet(name, name) {
  console.log(name, name);
}
greet("Rohit", "Mohit"); // Mohit Mohit
```

Problem: Both parameters have the same name. The last value passed overwrites previous ones. No error is thrown!

With Strict Mode:

```
"use strict";
function greet(name, name) { // ERROR: Duplicate parameter name not allowed
  console.log(name, name);
}
greet("Rohit", "Mohit");
```

Result: Immediate error → "Duplicate parameter name not allowed in this context"

How to Enable Strict Mode

Add this at the **very top** of your JavaScript file:

```
"use strict";
```

Key Benefits of Strict Mode

1. **Prevents silent errors:** Throws errors instead of silently failing
2. **Catches common mistakes:** Variable declarations, duplicate parameters, etc.
3. **Makes debugging easier:** Errors appear during development, not in production
4. **Follows modern JavaScript standards:** Uses latest ECMAScript rules strictly

Important Notes

- Modern JavaScript modules and classes use strict mode by default
 - You'll learn more about strict mode in backend/Node.js development
 - Always use strict mode for cleaner, more maintainable code
-

Global Objects in Different Environments

Understanding JavaScript Environments

JavaScript can run in different environments, each with its own global object.

Environment 1: Browser (Chrome, Firefox, etc.)

Global Object: `window`

```
console.log(window); // Prints the entire window object
```

What is the **window** object?

- The **global object** in browser environments
- Created by the **browser**, not JavaScript itself
- Part of **Web APIs**, not core JavaScript
- Contains: `document`, `setTimeout`, `setInterval`, `fetch`, etc.

Example:

```
console.log(window.document);  
console.log(window.setTimeout);  
console.log(window.fetch);
```

Environment 2: Node.js

Global Object: `global`

```
console.log(global); // Prints the global object in Node.js
```

What is the **global** object?

- The **global object** in Node.js environment
- Node.js doesn't know about `window` (browser-specific)
- Contains: `setTimeout`, `setInterval`, `clearTimeout`, `fetch`, `crypto`, etc.

Example:

```
console.log(global.setTimeout);  
console.log(global.fetch);  
console.log(global.crypto);
```

Important: `document` does NOT exist in Node.js

```
console.log(document); // ERROR: document is not defined
```

Why? Because `document` is part of browser Web APIs for DOM manipulation. Node.js has no DOM!

Key Differences: Browser vs Node.js

Feature	Browser	Node.js
---------	---------	---------

Global Object	<code>window</code>	<code>global</code>
DOM Access	✓ Yes (<code>document</code>)	✗ No
Web APIs	✓ Yes (full browser APIs)	✗ Limited
File System	✗ No	✓ Yes (<code>fs</code> module)
V8 Engine	✓ Yes	✓ Yes

The Universal Global: `globalThis`

Problem: Different environments have different names for the global object (`window`, `global`, etc.)

Solution: `globalThis`

```
console.log(globalThis); // Works in ALL environments
```

In Browser:

```
console.log(globalThis); // Points to `window`
```

In Node.js:

```
console.log(globalThis); // Points to `global`
```

Important Note About Global Objects

```
// Global Object is NOT part of core JavaScript
// It is provided by the environment (browser, Node.js, etc.)
// Each environment can name it differently
```

Visual Representation:

JavaScript Core



Environment adds Global Object



Browser → `window`

Node.js → global
Universal → globalThis

The **this** Keyword - Complete Breakdown

What is **this**?

this is a special keyword that refers to the **context** in which a function is executed. Its value depends on **how** and **where** the function is called.

Case 1: **this** in Global Scope

In Node.js Environment

```
console.log(this); // {} (empty object)
```

With Strict Mode:

```
"use strict";  
console.log(this); // {} (still empty object)
```

Why empty object? The reason relates to Node.js module system. This will be explained in backend development. For now, remember: In Node.js global scope, **this** points to an empty object.

In Browser Environment

```
console.log(this); // Window object
```

With Strict Mode:

```
"use strict";  
console.log(this); // Window object (no change)
```

Why Window object? In browser global scope, **this** always points to the global **window** object, regardless of strict mode.



Important Rule #1: Global Scope **this**

// In Node.js (global scope):

this → {} (empty object)

// In Browser (global scope):

this → window object

// Strict mode does NOT change this behavior in global scope

Case 2: **this** in Object Methods

When **this** is used inside an object's method, it refers to the **object that owns the method**.

Example 1: Basic Object Method

```
const user = {  
  name: "Rohit",  
  age: 39,  
  greet: function() {  
    console.log(this); // Points to `user` object  
    console.log(`Hi ${this.name}`); // Hi Rohit  
  }  
};  
  
user.greet();
```

Output:

```
{ name: 'Rohit', age: 39, greet: [Function: greet] }  
Hi Rohit
```

Why does **this** point to **user**?



Critical Concept: Function Invocation Rule

The golden rule: **this** points to **whoever invoked the function**

```
user.greet();  
// ↑  
// WHO invoked greet()? → user object  
// Therefore, `this` = user object
```

Example 2: Why Use **this** Instead of Direct Reference?

Bad Approach (Hard-coded):

```
const user = {
  name: "Rohit",
  greet: function() {
    console.log(`Hi ${user.name}`); // Hard-coded reference
  }
};
```

```
const user2 = {
  name: "Mohit",
  greet: user.greet // Copying the function
};
```

user2.greet(); // Hi Rohit (WRONG! We wanted Mohit)

Problem: Hard-coded **user.name** always refers to the first user, even when called from **user2**.

Good Approach (Using **this**):

```
const user = {
  name: "Rohit",
  greet: function() {
    console.log(`Hi ${this.name}`); // Dynamic reference
  }
};
```

```
const user2 = {
  name: "Mohit",
  greet: user.greet // Copying the function
};
```

user.greet(); // Hi Rohit
user2.greet(); // Hi Mohit (CORRECT!)

Why this works:

- When **user.greet()** is called → **this = user**

- When `user2.greet()` is called → `this = user2`

Important Rule #2: Object Method `this`

```
// When a method is called: object.method()
// Look at what's on the LEFT of the dot (.)
// That becomes `this` inside the function
```

```
object.method() → this = object
```

Case 3: `this` in Regular Functions (Independent Functions)

When a function is called independently (not as a method), behavior depends on strict mode.

Without Strict Mode

```
function greet() {
  console.log(this);
}
```

```
greet();
```

In Node.js:

```
// Points to global object
```

In Browser:

```
// Points to window object
```

Why? In non-strict mode, when no one explicitly invokes a function, JavaScript assumes it was invoked by the global object.

With Strict Mode

```
"use strict";
```

```
function greet() {
  console.log(this);
}
```

```
greet(); // undefined
```

Why **undefined**? Strict mode is more logical:

- No one explicitly invoked **greet()**
- Function called itself
- Therefore, **this** should be **undefined** (not randomly pointing to global object)



Important Rule #3: Independent Function **this**

// Regular function (not a method):

```
function func() {  
  console.log(this);  
}
```

// Without strict mode:

func() → this = global object (global/window)

// With strict mode:

"use strict";

func() → this = undefined

Case 4: **this** with **call**, **apply**, and **bind**

These methods allow you to **explicitly set** what **this** should refer to.

The Problem They Solve

```
function greet() {  
  console.log(`Hi ${this.name}`);  
}
```

```
const user = { name: "Rohit" };  
const user2 = { name: "Mohit" };  

```

```
greet(); // Hi undefined (this.name doesn't exist)
```

Goal: Make **greet()** work with different objects without rewriting the function for each object.

Using **call()**

```
function greet() {
```

```
    console.log(`Hi ${this.name}`);  
  }  
  
const user = { name: "Rohit", age: 30 };  
const user2 = { name: "Mohit", age: 25 };  
  
greet.call(user); // Hi Rohit  
greet.call(user2); // Hi Mohit
```

Syntax:

```
functionName.call(thisValue, arg1, arg2, ...)
```

What **call()** does:

1. Calls the function
2. Sets **this** to the object you pass as the first argument
3. Passes additional arguments to the function

Example with Parameters:

```
function incrementAge(value) {  
  this.age += value;  
  console.log(this.age);  
}  
  
const user = { name: "Rohit", age: 30 };  
  
incrementAge.call(user, 5); // 35  
incrementAge.call(user, 10); // 45
```

Using **apply()**

apply() works exactly like **call()**, but arguments are passed as an **array**.

```
function incrementAge(value, newName) {  
  this.age += value;  
  this.name = newName;  
  console.log(this.age, this.name);  
}  
  
const user2 = { name: "Mohit", age: 10 };
```

```
incrementAge.apply(user2, [10, "Mohan"]); // 20 Mohan
```

Syntax:

```
functionName.apply(thisValue, [arg1, arg2, ...])
```

Difference between `call()` and `apply()`:

```
// call() - arguments passed individually  
func.call(obj, arg1, arg2, arg3);
```

```
// apply() - arguments passed as array  
func.apply(obj, [arg1, arg2, arg3]);
```

Using `bind()`

`bind()` does NOT call the function immediately. Instead, it returns a **new function** with `this` permanently bound.

```
function incrementAge(value, newName) {  
  this.age += value;  
  this.name = newName;  
  console.log(this.age, this.name);  
}  
  
const user2 = { name: "Mohit", age: 10 };  
  
const incr = incrementAge.bind(user2, 10, "Mohan");  
  
console.log(incr); // [Function: bound incrementAge]  
  
incr(); // 20 Mohan (called later)
```

Syntax:

```
const boundFunc = functionName.bind(thisValue, arg1, arg2, ...);  
boundFunc(); // Call it whenever you want
```

Use Case: When you want to prepare a function with a specific context but call it later.

Important Rule #4: **call, apply, bind**

// All three methods let you explicitly set `this`:

// call() - immediate invocation, individual args
func.call(obj, arg1, arg2);

// apply() - immediate invocation, array of args
func.apply(obj, [arg1, arg2]);

// bind() - returns new function, call later
const bound = func.bind(obj, arg1, arg2);
bound();

Real-World Benefits

Memory Efficiency:

// BAD: Creating 100 copies of the same function
const user1 = { name: "User1", greet: function() {...} };
const user2 = { name: "User2", greet: function() {...} };
// ... 100 times = 100 function copies in memory!

// GOOD: One function, reused 100 times
function greet() {
 console.log(`Hi \${this.name}`);
}

const user1 = { name: "User1" };
const user2 = { name: "User2" };

greet.call(user1); // Reuse the same function
greet.call(user2); // Memory efficient!

Key Benefits:

1.  **Memory Efficiency:** One function definition, reused multiple times
2.  **DRY Principle:** Don't Repeat Yourself - write once, use everywhere
3.  **Flexibility:** Same function works with different objects

Case 5: **this** in Class Constructors

When you use the **new** keyword with a class or constructor function, **this** refers to the **newly created empty object**.

Example: Class Constructor

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
}
```

```
const p1 = new Person("Rohit", 20);  
console.log(p1); // Person { name: 'Rohit', age: 20 }
```

What Happens Behind the Scenes

// When you write:

```
const p1 = new Person("Rohit", 20);
```

// JavaScript does this:

// 1. Create empty object

```
const p1 = {};
```

// 2. Set `this` to point to that empty object

// 3. Run constructor code

```
this.name = "Rohit"; // p1.name = "Rohit"
```

```
this.age = 20;      // p1.age = 20
```

// 4. Return the object

```
return p1;
```

Step-by-step visualization:

```
// new Person("Rohit", 20)
```

Step 1: {}

Step 2: this → {}

Step 3: this.name = "Rohit" → { name: "Rohit" }

Step 4: this.age = 20 → { name: "Rohit", age: 20 }

Step 5: return object → Person { name: "Rohit", age: 20 }



Important Rule #5: Class Constructor **this**

```
class MyClass {  
  constructor() {  
    console.log(this); // newly created empty object  
  }  
}
```

```
// `new` keyword creates empty object  
// `this` points to that empty object  
// Properties added to `this` are added to that object
```

Case 6: **this** in Arrow Functions

Most Important Concept: Arrow functions do **NOT** have their own **this**. They inherit **this** from their **lexical environment** (surrounding scope).

What Does "No Own **this**" Mean?

```
// Regular function has its own `this`  
function regularFunc() {  
  console.log(this); // Has its own `this`  
}  
  
// Arrow function does NOT have its own `this`  
const arrowFunc = () => {  
  console.log(this); // Borrows `this` from outer scope  
};
```

Example 1: Arrow Function in Global Scope

In Node.js:

```
const greet = () => {  
  console.log(this);  
};  
  
greet(); // {} (empty object)
```

Why empty object?

- Arrow function has no own **this**
- Looks outside (lexical scope)

- Finds global scope `this`
- In Node.js global scope, `this = {}`

In Browser:

```
const greet = () => {
  console.log(this);
};
```

```
greet(); // Window object
```

Why Window?

- Arrow function has no own `this`
- Looks outside (lexical scope)
- Finds global scope `this`
- In browser global scope, `this = window`

Example 2: Arrow Function Inside Object Method

```
const user = {
  name: "Rohit",
  greet: () => {
    console.log(this);
  }
};
```

```
user.greet();
```

In Node.js: `{}` (empty object)

In Browser: `Window object`

Why doesn't `this` point to `user`?

```
// Arrow function has no own `this`
// Goes to outer scope
// Outer scope = global scope (not the `user` object!)
// Global scope `this` = empty object (Node.js) or window (browser)
```

Important Note: The `user` object is a **data structure**, not a **scope**. Arrow function looks for **scope**, not data structures.


```
// This is NOT a scope (it's just an object):  
const user = { ... }
```

```
// These ARE scopes:  
function func() { ... } // Function scope  
if (true) { ... }       // Block scope
```

Example 3: Arrow Function Inside Regular Function (The Solution!)

```
const user = {  
  name: "Rohit",  
  greet: function() {  
    console.log(this); // Points to `user`  
  
    const inner = () => {  
      console.log(this); // Also points to `user`!  
    };  
  
    inner();  
  }  
};  
  
user.greet();  
// Output:  
// { name: 'Rohit', greet: [Function: greet] }  
// { name: 'Rohit', greet: [Function: greet] }
```

Why does arrow function get **user**?

```
// Step 1: Arrow function has no own `this`  
const inner = () => { console.log(this); };
```

```
// Step 2: Look in outer scope (the `greet` function)  
greet: function() {  
  // This function has `this`  
  // And `this` points to `user` (because user.greet() was called)  
}
```

```
// Step 3: Arrow function borrows `this` from outer function  
// Therefore, arrow function's `this` = `user`
```



Important Rule #6: Arrow Function **this**

// Arrow functions inherit `this` from lexical environment:

```
const arrow = () => {  
  console.log(this); // Borrows from outer scope  
};
```

// LEXICAL ENVIRONMENT = surrounding scope where function is defined

// Arrow function looks for `this`:

// 1. In immediately surrounding scope

// 2. If not found, goes one level up

// 3. Continues until found or reaches global scope

Case 7: The Old Problem Arrow Functions Solved

The Classic Callback Problem

Before Arrow Functions (The Problem):

```
const user = {  
  name: "Rohit",  
  greet: function() {  
    function meet() {  
      console.log(this); // What is `this` here?  
    }  
    meet();  
  }  
};
```

user.greet();

Without Strict Mode:

// Prints: global object (global/window)

With Strict Mode:

"use strict";
// Prints: undefined

Why? The `meet()` function is called independently (no one explicitly invokes it). So `this` is either global object (non-strict) or `undefined` (strict).

Problem: We wanted `this` to be `user`, but it's not!

Old Solution (Before Arrow Functions)

```
const user = {
  name: "Rohit",
  greet: function() {
    const that = this; // Save reference to `this`

    function meet() {
      console.log(that.name); // Use `that` instead of `this`
    }
    meet();
  }
};

user.greet(); // Rohit
```

This worked, but it's a hacky workaround!

Modern Solution: Arrow Functions

```
const user = {
  name: "Rohit",
  greet: function() {
    const meet = () => {
      console.log(this.name); // Arrow function inherits `this`
    };
    meet();
  }
};

user.greet(); // Rohit
```

Why arrow functions are better:

- No need for `that = this` trick
- Naturally inherits `this` from outer scope
- Cleaner, more readable code

Real-World Use Case: Stopwatch with `setInterval`

Problem with Regular Functions

```
const stopwatch = {
  seconds: 0,
  start: function() {
    setInterval(function() {
      this.seconds++; // `this` doesn't point to stopwatch!
      console.log(this.seconds);
    }, 1000);
  }
};
```

stopwatch.start(); // NaN NaN NaN ...

Why NaN?

- `setInterval` invokes the callback
- Inside callback, `this` points to `Timeout` object (not `stopwatch`)
- `Timeout.seconds` is `undefined`
- `undefined++ = NaN`

Old Solution (Before Arrow Functions)

```
const stopwatch = {
  seconds: 0,
  start: function() {
    const that = this; // Save reference

    setInterval(function() {
      that.seconds++; // Use `that`
      console.log(that.seconds);
    }, 1000);
  }
};
```

stopwatch.start(); // 1 2 3 4 ...

Modern Solution: Arrow Functions

```
const stopwatch = {
  seconds: 0,
  start: function() {
    setInterval(() => {
      this.seconds++; // Arrow function inherits `this` from `start`
      console.log(this.seconds);
    });
  }
};
```

```
    }, 1000);  
  }  
};
```

```
stopwatch.start(); // 1 2 3 4 ...
```

How it works:

1. Arrow function has no own `this`
2. Looks in outer scope (`start` function)
3. `start`'s `this` points to `stopwatch` (because `stopwatch.start()` was called)
4. Arrow function borrows that `this`
5. Therefore, `this.seconds` correctly refers to `stopwatch.seconds`



Important Rule #7: Why Arrow Functions Were Created

// Arrow functions solve the callback `this` problem:

// OLD WAY (hacky):

```
const that = this;  
callback(function() {  
  that.property; // Use saved reference  
});
```

// NEW WAY (clean):

```
callback(() => {  
  this.property; // Arrow function inherits `this`  
});
```

Case 8: `this` in DOM Event Listeners

When you attach an event listener to a DOM element, `this` behavior depends on whether you use regular or arrow functions.

With Regular Function

```
<button id="firstBtn">Click Me</button>
```

```
<script>
```

```
const btn = document.getElementById("firstBtn");
```

```
btn.addEventListener("click", function() {
```

```
    console.log(this); // Points to the button element
  });
</script>
```

Output when clicked:

```
<button id="firstBtn">Click Me</button>
```

Why? The button element invoked the function, so `this` points to the button.

With Arrow Function

```
const btn = document.getElementById("firstBtn");

btn.addEventListener("click", () => {
  console.log(this); // Points to global object (window)
});
```

Output when clicked:

```
Window { ... }
```

Why? Arrow function has no own `this`, so it goes to outer scope (global scope), where `this = window`.



Important Rule #8: Event Listeners

// Regular function in event listener:

```
element.addEventListener("click", function() {
  this → element that was clicked
});
```

// Arrow function in event listener:

```
element.addEventListener("click", () => {
  this → outer scope (usually global/window)
});
```

Summary and Quick Reference

The Complete **this** Reference Table

Context	Strict Mode	Non-Strict Mode	Notes
Global Scope (Node.js)	<code>{}</code> (empty object)	<code>{}</code> (empty object)	Same in both modes
Global Scope (Browser)	<code>window</code>	<code>window</code>	Same in both modes
Object Method	Object that called it	Object that called it	Look at left of dot
Regular Function (independent)	<code>undefined</code>	Global object	Who invoked it?
Arrow Function	Inherited from outer scope	Inherited from outer scope	No own this
Class Constructor	New empty object	New empty object	Created by <code>new</code>
<code>call/apply/bind</code>	Object passed as arg	Object passed as arg	Explicit binding
Event Listener (regular)	DOM element	DOM element	Element invoked it
Event Listener (arrow)	Outer scope	Outer scope	Inherits from outer

The Golden Rules

Rule 1: Who Invoked It?

// Most important question: Who called the function?
object.method(); // `this` = object (left of the dot)

Rule 2: Arrow Functions Have No **this**

// Arrow functions inherit `this` from surrounding scope
const arrow = () => this; // Looks outside for `this`

Rule 3: `new` Creates Empty Object

// `new` keyword creates empty object and sets `this` to it
new MyClass(); // `this` = new empty object

Rule 4: Explicit Binding Wins

// call/apply/bind explicitly set `this`
func.call(obj); // `this` = obj (you decide)

Common Pitfalls and How to Avoid Them

Pitfall 1: Arrow Function in Object Method

// ❌ WRONG

```
const obj = {  
  name: "Rohit",  
  greet: () => {  
    console.log(this.name); // undefined (looks at global scope)  
  }  
};
```

// ✅ CORRECT

```
const obj = {  
  name: "Rohit",  
  greet: function() {  
    console.log(this.name); // Rohit (this = obj)  
  }  
};
```

Pitfall 2: Nested Function Losing **this**

// ❌ WRONG

```
const obj = {  
  name: "Rohit",  
  greet: function() {  
    function inner() {  
      console.log(this.name); // undefined or error  
    }  
    inner();  
  }  
};
```

// ✅ CORRECT (use arrow function)

```
const obj = {  
  name: "Rohit",  
  greet: function() {  
    const inner = () => {  
      console.log(this.name); // Rohit (inherits from greet)  
    };  
  }  
};
```



```
    inner();  
  }  
};
```

Pitfall 3: Event Listener with Arrow Function

```
// ❌ WRONG (if you need the element)  
button.addEventListener("click", () => {  
  console.log(this); // window (not the button!)  
});
```

```
// ✅ CORRECT (if you need the element)  
button.addEventListener("click", function() {  
  console.log(this); // button element  
});
```

Memory Aids

For Interviews:

1. Remember: "Left of the dot" for object methods
2. Remember: Arrow functions = "borrow from outside"
3. Remember: Independent functions = depends on strict mode
4. Remember: `call/apply/bind` = you explicitly set `this`

Visualization:

// Think of `this` as asking: "Who am I working for?"

```
user.method(); // "I work for `user`"  
func();        // "Nobody called me" → undefined (strict) or global (non-strict)  
() => {}        // "I borrow my boss from outside"  
func.call(obj) // "You told me to work for `obj`"
```

Final Notes

When to Use Each Approach

Regular Functions:

- Use when you need dynamic `this` (object methods, constructors)

- Use in event listeners when you need the element reference

Arrow Functions:

- Use in callbacks where you want to preserve outer `this`
- Use in array methods (`map`, `filter`, etc.)
- Use when you don't care about `this` at all

`call`/`apply`/`bind`:

- Use when you want to reuse a function with different objects
- Use `call`/`apply` for immediate invocation
- Use `bind` when you want to prepare a function for later use

Review Strategy

Before your interview, review:

1. ☒ Global scope `this` (Node.js vs Browser)
2. ☒ Object method `this` (left of the dot)
3. ☒ Independent function `this` (strict vs non-strict)
4. ☒ Arrow function `this` (lexical inheritance)
5. ☒ `call`/`apply`/`bind` differences
6. ☒ Common pitfalls and solutions

Practice Exercises

Try to predict the output:

```
"use strict";
```

```
const obj = {  
  name: "Rohit",  
  regular: function() {  
    console.log(this.name);  
  },  
  arrow: () => {  
    console.log(this.name);  
  },  
  nested: function() {  
    const inner = () => {  
      console.log(this.name);  
    };  
    inner();  
  }  
};
```

```
}  
};
```

```
obj.regular(); // ?  
obj.arrow(); // ?  
obj.nested(); // ?
```

```
const func = obj.regular;  
func(); // ?
```

Conclusion

Understanding **this**, **call**, **apply**, and **bind** is crucial for:

-  Passing technical interviews
-  Writing maintainable code
-  Understanding how JavaScript works under the hood
-  Debugging complex issues

Remember: These concepts might seem complicated at first, but with practice and by referring back to these notes, they'll become second nature. Good luck with your interviews! 🚀

Here's the complete fixed code:

```
"use strict";
```

```
const obj = {  
  name: "Rohit",  
  regular: function() {  
    console.log(this.name);  
  },  
  arrow: () => {  
    console.log(this.name);  
  },  
  nested: function() {  
    const inner = () => {  
      console.log(this.name);  
    };  
    inner();  
  }  
}
```

```
};
```

```
obj.regular(); // ?
```

```
obj.arrow(); // ?
```

```
obj.nested(); // ?
```

```
const func = obj.regular;
```

```
func(); // ?
```

Answers and Explanations:

1. `obj.regular()`

Output: "Rohit"

Explanation:

- Regular function inside object method
 - Called as `obj.regular()` → left of the dot is `obj`
 - Therefore, `this = obj`
 - `this.name = "Rohit"`
-

2. `obj.arrow()`

Output: `undefined`

Explanation:

- Arrow function has no own `this`
- Looks in outer scope (lexical environment)
- Outer scope is global scope (the object `{}` is NOT a scope)
- In strict mode, global `this` in Node.js = `{}`
- Empty object has no `name` property
- Therefore, `this.name = undefined`

In Browser: Would print `undefined` as well (window object has no `name` property by default)

3. `obj.nested()`

Output: "Rohit"

Explanation:

- `nested` is a regular function, so when called as `obj.nested()`, its `this = obj`
 - Inside `nested`, `inner` is an arrow function
 - Arrow function inherits `this` from outer scope
 - Outer scope is the `nested` function, where `this = obj`
 - Therefore, arrow function's `this = obj`
 - `this.name = "Rohit"`
-

4. `func()`

Output: `TypeError: Cannot read property 'name' of undefined`

Explanation:

- `func` is a reference to `obj.regular` (the function itself)
- When called as `func()` (not as `obj.regular()`), it's called independently
- In strict mode, independent function calls have `this = undefined`
- Trying to access `undefined.name` throws a `TypeError`

Without Strict Mode: Would print `undefined` (because `this` would be global object, which has no `name` property)

Complete Working Example with Output:

```
"use strict";
```

```
const obj = {  
  name: "Rohit",  
  regular: function() {  
    console.log(this.name);  
  },  
  arrow: () => {  
    console.log(this.name);  
  },  
};
```

```

    nested: function() {
      const inner = () => {
        console.log(this.name);
      };
      inner();
    }
  };

console.log("\n1. obj.regular():");
obj.regular(); // "Rohit"

console.log("\n2. obj.arrow():");
obj.arrow(); // undefined

console.log("\n3. obj.nested():");
obj.nested(); // "Rohit"

console.log("\n4. func():");
const func = obj.regular;
try {
  func(); // TypeError
} catch(error) {
  console.log("Error:", error.message);
}

```

Expected Console Output:

```

1. obj.regular():
Rohit

2. obj.arrow():
undefined

3. obj.nested():
Rohit

4. func():
Error: Cannot read property 'name' of undefined

```

How to Fix #4 (Different Solutions):

Solution 1: Use `call()`

```
const func = obj.regular;  
func.call(obj); // "Rohit"
```

Solution 2: Use `bind()`

```
const func = obj.regular.bind(obj);  
func(); // "Rohit"
```

Solution 3: Call it as a method

```
obj.regular(); // "Rohit"
```

Solution 4: Convert to arrow function (if appropriate)

```
const obj = {  
  name: "Rohit",  
  regular: function() {  
    const safeFn = () => {  
      console.log(this.name);  
    };  
    return safeFn;  
  }  
};
```

```
const func = obj.regular();  
func(); // "Rohit"
```